

IssueForge

Overview

IssueForge is an AI-powered Drupal issue analysis and reproduction platform.

Its goal is to reduce the time required for developers to understand, reproduce, validate, and fix Drupal core and contributed module issues from Drupal.org.

Today, a developer typically spends significant time manually:

- Reading the issue summary
- Reading dozens of comments
- Determining affected subsystems
- Understanding root cause
- Finding the latest valid patch
- Setting up the correct Drupal version
- Enabling required modules
- Reproducing the issue
- Understanding where code changes are needed

IssueForge automates most of this process.

Problem Statement

Drupal.org contains thousands of issues.

A typical issue can contain:

- Long descriptions
- Multiple patches
- Many comment threads
- Several revisions of proposed fixes
- Different target Drupal versions
- Complex reproduction steps

Developers spend more time understanding the issue than actually fixing it.

This causes:

- Slow onboarding
- High context switching costs
- Duplicate investigation work

- Difficulty contributing to Drupal core
- Increased bug resolution time

IssueForge aims to convert a Drupal issue URL into an actionable engineering plan.

Vision

Input:

Drupal.org issue URL

Example:

<https://www.drupal.org/project/drupal/issues/3517198>

Output:

A complete engineering context package including:

- Problem summary
 - Expected behavior
 - Actual behavior
 - Proposed resolution
 - Affected files
 - Affected functions
 - Affected subsystems
 - Root cause signals
 - Fix strategies
 - Patch status
 - Environment setup plan
 - Reproduction steps
 - Patch application plan
 - Future AI-generated fix recommendations
-

Current Architecture

Data Collection Layer

DrupalAPIClient

Responsible for:

- Fetching issue metadata
- Fetching issue descriptions
- Fetching patch references
- Fetching comments

Produces:

```
{
  "title": "...",
  "component": "...",
  "version": "...",
  "patch_file_ids": [...],
  "comment_ids": [...]}
}
```

Patch Analysis Layer

DrupalPatchClient

Downloads patch files from Drupal.org.

PatchAnalyzer

Extracts:

- Modified files
- Modified functions
- Modified methods

Example:

```
{
  "modified_files": [
    "core/modules/views/src/ViewExecutable.php"
  ],
}
```

```
"modified_functions": [  
  "_build",  
  "convertExposedInput"  
]  
}
```

MultiPatchAnalyzer

Responsible for:

- Analyzing all available patches
- Comparing patch revisions
- Selecting best/latest patch

Current strategy:

Latest patch wins.

Future strategy:

Highest quality patch wins.

Issue Understanding Layer

IssueDescriptionParser

Extracts:

- Problem summary
 - Reproduction steps
 - Expected behavior
 - Actual behavior
 - Proposed resolution
-

CommentSignalDetector

Analyzes issue comments.

Detects:

- RTBC indicators
- Review requests

- Maintainer involvement
- Test failures
- Patch revisions

Example:

```
{
  "comment_signals": [
    "Ready for community review"
  ]
}
```

PatchStatusClassifier

Converts comment signals into lifecycle states.

Examples:

- needs_work
- needs_review
- rtbc
- fixed

Code Intelligence Layer

SubsystemDetector

Detects Drupal subsystems.

Examples:

- Views
- Entity API
- Plugin System
- Routing
- Form API
- Theme System

RootCauseDetector

Infers likely root causes.

Examples:

- Query construction issue
 - Plugin lifecycle issue
 - Filter normalization issue
 - Access control issue
-

FixStrategyGenerator

Generates suggested engineering directions.

Examples:

- Refactor query condition aggregation
 - Normalize filter inputs
 - Adjust plugin lifecycle handling
-

PatchPlanGenerator

Produces an engineering plan.

Example:

```
{
  "target_files": [...],
  "target_functions": [...],
  "risk_level": "high"
}
```

DiffSkeletonGenerator

Generates patch scaffolding.

Example:

```
@@ function convertExposedInput @@
+ // TODO
```

This provides a starting point for implementation.

Environment Automation Layer

EnvironmentPlanner

Generates a reproducible Drupal environment.

Detects:

- Drupal version
- Required modules
- Required themes
- Install profile
- PHP version

Example:

```
{  
  "checkout_ref": "11.x",  
  "project_type": "drupal11",  
  "php_version": "8.2"  
}
```

VersionResolver

Converts Drupal issue versions into:

- Git branch
- PHP version
- DDEV project type

Example:

```
main → 11.x  
11.x-dev → 11.x  
10.3.x-dev → 10.3.x
```

DdevScriptGenerator

Generates complete environment setup scripts.

Example:

```
git clone Drupal
git checkout 11.x
ddev start
composer install
drush si
```

PatchApplyScriptGenerator

Generates patch application scripts.

Important:

Patch is NOT automatically applied.

IssueForge intentionally separates:

1. Environment creation
2. Patch application

This allows user confirmation before patch installation.

Reproduction Layer

Reproduction Step Extraction

IssueForge extracts reproduction steps directly from issue descriptions.

Example:

1. Create View
 2. Add exposed filter
 3. Enable grouped filters
 4. Test multi-selection
-

ReproductionScriptGenerator

Converts extracted steps into reproducible workflows.

Current version:

Manual guidance.

Future version:

Automated Drush and browser actions.

What Has Already Been Built

Completed:

✓ Drupal issue metadata extraction

✓ Comment extraction

✓ Patch download

✓ Patch analysis

✓ Multi-patch support

✓ Function detection

✓ File detection

✓ Subsystem detection

✓ Root cause detection

✓ Fix strategy generation

✓ Comment signal detection

✓ Patch lifecycle classification

✓ Patch planning

✓ Diff skeleton generation

✓ Version resolution

✓ Environment planning

- ✓ DDEV script generation
 - ✓ Patch application script generation
 - ✓ Reproduction step extraction
 - ✓ Reproduction script generation
 - ✓ MCP-compatible API endpoint
-

Current API Output

Input:

Drupal Issue URL

Output:

```
{
  "issue_title": "...",
  "problem_summary": "...",
  "modified_files": [...],
  "root_cause_signals": [...],
  "patch_status": "...",
  "environment_plan": {...},
  "ddev_script": "...",
  "patch_plan": {...}
}
```

Next Major Milestones

Phase 1

Patch Compatibility Detection

Goal:

Determine whether latest patch actually matches target branch.

Phase 2

Theme Awareness

Detect:

- Core themes
- Contributed themes
- Theme-specific issues

Generate theme-aware environments.

Phase 3

Automatic Environment Creation

Instead of generating scripts:

IssueForge directly executes:

```
ddev start
composer install
drush si
```

and provisions environment automatically.

Phase 4

Patch Application Approval Workflow

IssueForge should ask:

"Environment is ready. Latest patch 7085307 is available. Would you like me to apply it?"

Only after confirmation:

Apply patch.

Phase 5

Automated Issue Reproduction

Convert reproduction steps into:

- Drush commands
- Config imports
- Browser automation

Goal:

One-click issue reproduction.

Phase 6

AI Fix Generation

IssueForge analyzes:

- Issue description
- Comments
- Patch history
- Modified files

Then generates:

- Suggested code changes
 - Candidate patch
 - Test coverage suggestions
-

Phase 7

Automated Validation

IssueForge automatically:

- Applies patch
 - Runs PHPUnit
 - Runs Kernel tests
 - Runs Functional tests
 - Generates validation report
-

Long-Term Vision

IssueForge becomes an AI-powered Drupal engineering assistant.

A developer provides:

Drupal Issue URL

IssueForge performs:

1. Analyze issue
2. Build environment
3. Reproduce issue
4. Apply patch (with approval)
5. Validate patch
6. Suggest improvements
7. Generate fix candidates
8. Create merge-ready patch

The objective is to reduce Drupal issue investigation time from hours to minutes while improving contribution quality and onboarding efficiency.